1.) Ihr Script soll von ihrer Ubuntu VM eine BGP-Konfiguration zu ihrem CSR und N9kV pushen.
2.) Die BGP-Konfiguration baut je eine iBGP Session zum CSR_Pod_7 und NX_Pod_7 auf, die als redundante Route Reflektoren konfiguriert sind. Die ASN für BGP ist 65000 und als RID soll die lokale Loopback0-Adresse verwendet werden. Der Prefix auf dem lokalen Loopback1-Interface soll per BGP verbreitet werden.
3.) Deaktivieren Sie (über das CLI) auf ihrem CSR und N9kv OSPF auf dem Interface Loopback1.
4.) Öffnen sie eine .py Datei im Sublime, um das Script zu erstellen:

```
et@ubuntu:~ $ subl <Datei-Name>.py
```

5.) Importieren Sie das Paket netmiko:

```
import netmiko
```

6.) Legen Sie Dictionaries mit den Verbindungsdaten für ihren CSR und N9kv an:

```
conn_data_ios = {
        'ip' : '10.196.0.x',
        'username' : 'student',
        'password' : '1234QWer',
        'device_type' : 'cisco_ios_telnet'
}

conn_data_nx = {
        'ip' : '10.196.0.x',
        'username' : 'admin',
        'password' : '1234QWer',
        'device_type' : 'cisco_nxos'
}
```

(x ist jeweils geeignet zu wählen, also z.B. 201 für den CSR_Pod_1 und 221 für den NX_Pod_1)

7.) Hinterlegen Sie die BGP-Konfiguration für den CSR und N9Kv in zwei separaten Listen:

```
bgp_ios = [
'router bgp 65000',
'router-id 100.255.255.x',
'network 100.0.x.0 mask 255.255.255.0',
'neighbor 100.255.255.7 remote-as 65000',
'neighbor 100.255.255.7 update-source loopback0',
'neighbor 100.255.255.7 next-hop-self',
'neighbor 200.255.255.7 remote-as 65000',
'neighbor 200.255.255.7 update-source loopback0',
'neighbor 200.255.255.7 next-hop-self',
]

bgp_nx = [
```

```
'router bgp 65000',
'router-id 200.255.255.x',
'address-family ipv4 unicast',
'network 200.0.x.0/24',
'neighbor 100.255.255.7 remote-as 65000',
'update-source loopback0',
'address-family ipv4 unicast',
'next-hop-self',
'neighbor 200.255.255.7 remote-as 65000',
'update-source loopback0',
'address-family ipv4 unicast',
'next-hop-self',
]
```

(Auch hier muss für x ihre Pod-Nummer eingesetzt werden.)

8.) Erzeugen Sie mit der Factory Function ConnectHandler() je ein Objekt zum Aufbau der Verbindungen zum CSR und N9Kv. Die in den Listen befindliche Konfiguration soll gepusht und einige Kommandos zur Verifikation gesendet werden, bevor die Verbindung abgebaut wird:

```
session_ios = netmiko.ConnectHandler(**conn_data_ios)
session_ios.send_config_set(config_commands = bgp_ios)
print(session_ios.send_command('show running | section bgp'))
print(session_ios.send_command('show bgp ipv4 unicast summary'))
session_ios.disconnect()

session_nx = netmiko.ConnectHandler(**conn_data_nx)
session_nx.send_config_set(config_commands = 'feature bgp')
session_nx.send_config_set(config_commands = bgp_nx)
print(session_nx.send_command('show running bgp'))
print(session_nx.send_command('show bgp ipv4 unicast summary'))
session_nx.disconnect()
```

(ACHTUNG: Wenn Sie für die Listen und Dictionaries andere Namen gewählt haben, dann müssen Sie die obigen Anweisungen entsprechend modifizieren.)

9.) Ihr Script ist nun betriebsbereit. Speichern Sie es in Sublime mit Str-S ab und rufen Sie es über die Shell unter Python 3.7 auf:

```
et@ubuntu:~ $ python3.7 <Datei-Name>.py
```

10.) Verifizieren Sie von ihrem CSR und N9kv die Erreichbarkeit der Loopback1-Adressen der anderen Pods.